

Projeto Completo — CRUD de Alunos com Administrador, PostgreSQL e JWT

1. Objetivo

O sistema possui dois módulos:

```
modules/  
├─ admin/  
└─ aluno/
```

O administrador faz cadastro inicial, login, recebe um JWT e utiliza o CRUD de alunos. O aluno não faz login; ele é apenas um registro administrado.

```
Administrador  
├─ cadastra alunos  
├─ lista alunos  
├─ edita alunos  
└─ exclui alunos
```

2. Fluxo geral

```
Primeiro acesso  
↓  
Cadastro do administrador  
↓  
Senha criptografada com bcrypt  
↓  
Administrador salvo no PostgreSQL  
↓  
Login  
↓  
JWT  
↓  
Authorization: Bearer TOKEN  
↓  
Middleware valida o token  
↓  
CRUD de alunos liberado
```

3. Estrutura do backend

```
backend/  
├── src/  
│   ├── config/  
│   │   └── database.js  
│   ├── middlewares/  
│   │   └── autenticacao.middleware.js  
│   ├── modules/  
│   │   ├── admin/  
│   │   │   ├── controllers/  
│   │   │   │   └── admin.controller.js  
│   │   │   ├── models/  
│   │   │   │   └── admin.model.js  
│   │   │   └── routes/  
│   │   │       └── admin.route.js  
│   │   └── aluno/  
│   │       ├── controllers/  
│   │       │   └── aluno.controller.js  
│   │       ├── models/  
│   │       │   └── aluno.model.js  
│   │       └── routes/  
│   │           └── aluno.route.js  
│   └── index.js  
├── .env  
├── .env.example  
├── .gitignore  
└── package.json
```

4. Dependências

```
npm install express pg dotenv bcryptjs jsonwebtoken cors  
npm install -D nodemon
```

Exemplo de scripts no `package.json`:

```
{  
  "type": "module",  
  "scripts": {  
    "dev": "nodemon src/index.js",  
    "start": "node src/index.js"  
  }  
}
```

5. Banco de dados

5.1 Tabela de administradores

```
CREATE TABLE admins (  
  id SERIAL PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  senha VARCHAR(255) NOT NULL,  
  ativo BOOLEAN NOT NULL DEFAULT TRUE,  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

5.2 Tabela de alunos

```
CREATE TABLE alunos (  
  matricula VARCHAR(9) PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE  
);
```

O aluno não precisa de senha porque não acessa o sistema.

6. Variáveis de ambiente

.env

```
PORTA=3000  
  
PGUSER=postgres  
PGPASSWORD=sua_senha  
PGHOST=localhost  
PGPORT=5432  
PGDATABASE=crud_alunos  
  
JWT_SECRET=uma_chave_secreta_grande_e_dificil  
JWT_TEMPO_EXPIRACAO=1h
```

.env.example

```
PORTA=  
PGUSER=  
PGPASSWORD=  
PGHOST=  
PGPORT=  
PGDATABASE=
```

```
JWT_SECRET=  
JWT_TEMPO_EXPIRACAO=
```

.gitignore

```
node_modules  
.env
```

7. Conexão com PostgreSQL

Arquivo: `src/config/database.js`

```
import pg from "pg";  
import dotenv from "dotenv";  
  
dotenv.config();  
  
const { Pool } = pg;  
  
const conexao = new Pool({  
  user: process.env.PGUSER,  
  password: process.env.PGPASSWORD,  
  host: process.env.PGHOST,  
  port: process.env.PGPORT,  
  database: process.env.PGDATABASE  
});  
  
try {  
  await conexao.query("SELECT 1");  
  console.log("Conectado ao PostgreSQL com sucesso.");  
} catch (erro) {  
  console.log("Erro ao conectar ao PostgreSQL.");  
  console.log(erro.message);  
}  
  
export default conexao;
```

8. Model do administrador

Arquivo: `src/modules/admin/models/admin.model.js`

```
import conexao from "../../../config/database.js";  
  
class AdminModel {  
  static async contarAdmins() {
```

```
    const resultado = await conexao.query(
      "SELECT COUNT(*) FROM admins"
    );

    return Number(resultado.rows[0].count);
  }

  static async buscarPorEmail(email) {
    const resultado = await conexao.query(
      `
      SELECT id, nome, email, senha, ativo, criado_em
      FROM admins
      WHERE email = $1
      `,
      [email]
    );

    return resultado.rows[0];
  }

  static async buscarPorId(id) {
    const resultado = await conexao.query(
      `
      SELECT id, nome, email, ativo, criado_em
      FROM admins
      WHERE id = $1
      `,
      [id]
    );

    return resultado.rows[0];
  }

  static async cadastrar(nome, email, senha) {
    const resultado = await conexao.query(
      `
      INSERT INTO admins (nome, email, senha)
      VALUES ($1, $2, $3)
      RETURNING id, nome, email, ativo, criado_em
      `,
      [nome, email, senha]
    );

    return resultado.rows[0];
  }
}

export default AdminModel;
```

9. Controller do administrador

Arquivo: `src/modules/admin/controllers/admin.controller.js`

```
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import AdminModel from "../models/admin.model.js";

class AdminController {
  static async cadastrar(requisicao, resposta) {
    try {
      const { nome, email, senha } = requisicao.body;

      if (!nome || !email || !senha) {
        return resposta.status(400).json({
          mensagem: "Nome, e-mail e senha são obrigatórios."
        });
      }

      if (senha.length < 6) {
        return resposta.status(400).json({
          mensagem: "A senha deve possuir pelo menos 6 caracteres."
        });
      }

      const quantidadeAdmins = await AdminModel.contarAdmins();

      if (quantidadeAdmins > 0) {
        return resposta.status(403).json({
          mensagem: "O administrador inicial já foi cadastrado."
        });
      }

      const adminExistente = await AdminModel.buscarPorEmail(email);

      if (adminExistente !== undefined) {
        return resposta.status(409).json({
          mensagem: "Já existe um administrador com este e-mail."
        });
      }

      const senhaCriptografada = await bcrypt.hash(senha, 10);

      const admin = await AdminModel.cadastrar(
        nome,
        email,
        senhaCriptografada
      );

      return resposta.status(201).json({
        mensagem: "Administrador cadastrado com sucesso.",
        admin
      });
    } catch (erro) {
      console.log(erro);
    }
  }
}
```

```
    if (erro.code === "23505") {
      return resposta.status(409).json({
        mensagem: "Este e-mail já está cadastrado."
      });
    }

    return resposta.status(500).json({
      mensagem: "Erro ao cadastrar administrador."
    });
  }
}

static async login(requisicao, resposta) {
  try {
    const { email, senha } = requisicao.body;

    if (!email || !senha) {
      return resposta.status(400).json({
        mensagem: "E-mail e senha são obrigatórios."
      });
    }

    const admin = await AdminModel.buscarPorEmail(email);

    if (admin === undefined) {
      return resposta.status(401).json({
        mensagem: "E-mail ou senha inválidos."
      });
    }

    if (admin.ativo === false) {
      return resposta.status(403).json({
        mensagem: "Este administrador está desativado."
      });
    }

    const senhaCorreta = await bcrypt.compare(senha, admin.senha);

    if (senhaCorreta === false) {
      return resposta.status(401).json({
        mensagem: "E-mail ou senha inválidos."
      });
    }

    const token = jwt.sign(
      {
        id: admin.id,
        nome: admin.nome,
        email: admin.email
      },
      process.env.JWT_SECRET,
      {
        expiresIn: process.env.JWT_TEMPO_EXPIRACAO || "1h"
      }
    );
  }
}
```

```
        }
    });

    return resposta.status(200).json({
        mensagem: "Login realizado com sucesso.",
        token,
        admin: {
            id: admin.id,
            nome: admin.nome,
            email: admin.email
        }
    });
} catch (erro) {
    console.log(erro);

    return resposta.status(500).json({
        mensagem: "Erro ao realizar login."
    });
}
}

static async perfil(requisicao, resposta) {
    try {
        const admin = await AdminModel.buscarPorId(
            requisicao.admin.id
        );

        if (admin === undefined) {
            return resposta.status(404).json({
                mensagem: "Administrador não encontrado."
            });
        }

        return resposta.status(200).json({ admin });
    } catch (erro) {
        console.log(erro);

        return resposta.status(500).json({
            mensagem: "Erro ao buscar o perfil."
        });
    }
}
}

export default AdminController;
```

10. Middleware de autenticação JWT

Arquivo: `src/middlewares/autenticacao.middleware.js`


```
import jwt from "jsonwebtoken";

function verificarToken(requisicao, resposta, proximo) {
  const autorizacao = requisicao.headers.authorization;

  if (!autorizacao) {
    return resposta.status(401).json({
      mensagem: "Token não informado."
    });
  }

  const partes = autorizacao.split(" ");

  if (partes.length !== 2) {
    return resposta.status(401).json({
      mensagem: "Formato do token inválido."
    });
  }

  const tipo = partes[0];
  const token = partes[1];

  if (tipo !== "Bearer" || !token) {
    return resposta.status(401).json({
      mensagem: "Token inválido."
    });
  }

  try {
    const dadosToken = jwt.verify(
      token,
      process.env.JWT_SECRET
    );

    requisicao.admin = dadosToken;
    proximo();
  } catch (erro) {
    return resposta.status(401).json({
      mensagem: "Token inválido ou expirado."
    });
  }
}

export default verificarToken;
```

Após a validação, os controllers seguintes podem usar:

```
requisicao.admin.id
requisicao.admin.nome
```

```
requisicao.admin.email
```

11. Rotas do administrador

Arquivo: `src/modules/admin/routes/admin.route.js`

```
import express from "express";
import AdminController from "../../controllers/admin.controller.js";
import verificarToken from "../../../middlewares/autenticacao.middleware.js";

const router = express.Router();

router.post(
  "/admin/cadastrar",
  AdminController.cadastrar
);

router.post(
  "/admin/login",
  AdminController.login
);

router.get(
  "/admin/perfil",
  verificarToken,
  AdminController.perfil
);

export default router;
```

12. Model do aluno

Arquivo: `src/modules/aluno/models/aluno.model.js`

```
import conexao from "../../../config/database.js";

class AlunoModel {
  static async listarTodos() {
    const resultado = await conexao.query(
      `
      SELECT matricula, nome, email
      FROM alunos
      ORDER BY nome
      `
    );

    return resultado.rows;
  }
}
```

```
}

static async listarPorMatricula(matricula) {
  const resultado = await conexao.query(
    `
    SELECT matricula, nome, email
    FROM alunos
    WHERE matricula = $1
    `,
    [matricula]
  );

  return resultado.rows[0];
}

static async cadastrar(matricula, nome, email) {
  const resultado = await conexao.query(
    `
    INSERT INTO alunos (matricula, nome, email)
    VALUES ($1, $2, $3)
    RETURNING matricula, nome, email
    `,
    [matricula, nome, email]
  );

  return resultado.rows[0];
}

static async editarTotal(matricula, nome, email) {
  const resultado = await conexao.query(
    `
    UPDATE alunos
    SET nome = $1, email = $2
    WHERE matricula = $3
    RETURNING matricula, nome, email
    `,
    [nome, email, matricula]
  );

  return resultado.rows[0];
}

static async editarParcial(matricula, campos, valores) {
  const comandos = [];

  for (let indice = 0; indice < campos.length; indice++) {
    comandos.push(
      campos[indice] + " = $" + (indice + 1)
    );
  }

  valores.push(matricula);

  const consulta = `
```

```
        UPDATE alunos
        SET ${comandos.join(", ")}
        WHERE matricula = ${valores.length}
        RETURNING matricula, nome, email
    `;

    const resultado = await conexao.query(consulta, valores);
    return resultado.rows[0];
}

static async excluirPorMatricula(matricula) {
    const resultado = await conexao.query(
        `
        DELETE FROM alunos
        WHERE matricula = $1
        RETURNING matricula, nome, email
        `,
        [matricula]
    );

    return resultado.rows[0];
}

static async excluirTodos() {
    const resultado = await conexao.query(
        `
        DELETE FROM alunos
        RETURNING matricula, nome, email
        `
    );

    return resultado.rows;
}
}

export default AlunoModel;
```

13. Controller do aluno

Arquivo: `src/modules/aluno/controllers/aluno.controller.js`

```
import AlunoModel from "../../models/aluno.model.js";

class AlunoController {
    static async listarTodos(requisicao, resposta) {
        try {
            const alunos = await AlunoModel.listarTodos();
            return resposta.status(200).json(alunos);
        } catch (erro) {
            console.log(erro);
        }
    }
}
```

```
        return resposta.status(500).json({
            mensagem: "Erro ao listar alunos."
        });
    }
}

static async listarPorMatricula(requisicao, resposta) {
    try {
        const aluno = await AlunoModel.listarPorMatricula(
            requisicao.params.matricula
        );

        if (aluno === undefined) {
            return resposta.status(404).json({
                mensagem: "Aluno não encontrado."
            });
        }

        return resposta.status(200).json(aluno);
    } catch (erro) {
        console.log(erro);
        return resposta.status(500).json({
            mensagem: "Erro ao buscar aluno."
        });
    }
}

static async cadastrar(requisicao, resposta) {
    try {
        const { matricula, nome, email } = requisicao.body;

        if (!matricula || !nome || !email) {
            return resposta.status(400).json({
                mensagem: "Matrícula, nome e e-mail são obrigatórios."
            });
        }

        const aluno = await AlunoModel.cadastrar(
            matricula,
            nome,
            email
        );

        return resposta.status(201).json({
            mensagem: "Aluno cadastrado com sucesso.",
            aluno
        });
    } catch (erro) {
        console.log(erro);

        if (erro.code === "23505") {
            return resposta.status(409).json({
                mensagem: "Matrícula ou e-mail já cadastrado."
            });
        }
    }
}
```

```
    }

    return resposta.status(500).json({
      mensagem: "Erro ao cadastrar aluno."
    });
  }
}

static async editarTotal(requisicao, resposta) {
  try {
    const { nome, email } = requisicao.body;
    const { matricula } = requisicao.params;

    if (!nome || !email) {
      return resposta.status(400).json({
        mensagem: "Nome e e-mail são obrigatórios."
      });
    }

    const aluno = await AlunoModel.editarTotal(
      matricula,
      nome,
      email
    );

    if (aluno === undefined) {
      return resposta.status(404).json({
        mensagem: "Aluno não encontrado."
      });
    }

    return resposta.status(200).json({
      mensagem: "Aluno atualizado com sucesso.",
      aluno
    });
  } catch (erro) {
    console.log(erro);
    return resposta.status(500).json({
      mensagem: "Erro ao editar aluno."
    });
  }
}

static async editarParcial(requisicao, resposta) {
  try {
    const { matricula } = requisicao.params;
    const camposPermitidos = ["nome", "email"];
    const campos = [];
    const valores = [];

    for (const campo of camposPermitidos) {
      if (requisicao.body[campo] !== undefined) {
        campos.push(campo);
        valores.push(requisicao.body[campo]);
      }
    }
  }
}
```

```
    }
  }

  if (campos.length === 0) {
    return resposta.status(400).json({
      mensagem: "Informe pelo menos um campo para atualizar."
    });
  }

  const aluno = await AlunoModel.editarParcial(
    matricula,
    campos,
    valores
  );

  if (aluno === undefined) {
    return resposta.status(404).json({
      mensagem: "Aluno não encontrado."
    });
  }

  return resposta.status(200).json({
    mensagem: "Aluno atualizado parcialmente.",
    aluno
  });
} catch (erro) {
  console.log(erro);
  return resposta.status(500).json({
    mensagem: "Erro ao editar aluno."
  });
}
}

static async excluirPorMatricula(requisicao, resposta) {
  try {
    const aluno = await AlunoModel.excluirPorMatricula(
      requisicao.params.matricula
    );

    if (aluno === undefined) {
      return resposta.status(404).json({
        mensagem: "Aluno não encontrado."
      });
    }

    return resposta.status(200).json({
      mensagem: "Aluno excluído com sucesso.",
      aluno
    });
  } catch (erro) {
    console.log(erro);
    return resposta.status(500).json({
      mensagem: "Erro ao excluir aluno."
    });
  }
}
```

```
    }  
  }  
  
  static async excluirTodos(requisicao, resposta) {  
    try {  
      const alunos = await AlunoModel.excluirTodos();  
  
      return resposta.status(200).json({  
        mensagem: "Todos os alunos foram excluídos.",  
        quantidade: alunos.length  
      });  
    } catch (erro) {  
      console.log(erro);  
      return resposta.status(500).json({  
        mensagem: "Erro ao excluir os alunos."  
      });  
    }  
  }  
}  
  
export default AlunoController;
```

14. Rotas protegidas do aluno

Arquivo: `src/modules/aluno/routes/aluno.route.js`

```
import express from "express";  
import AlunoController from "../controllers/aluno.controller.js";  
import verificarToken from "../../../middlewares/autenticacao.middleware.js";  
  
const router = express.Router();  
  
router.get(  
  "/listar",  
  verificarToken,  
  AlunoController.listarTodos  
);  
  
router.get(  
  "/listar/:matricula",  
  verificarToken,  
  AlunoController.listarPorMatricula  
);  
  
router.post(  
  "/cadastrar",  
  verificarToken,  
  AlunoController.cadastrar  
);
```



```
router.put(
  "/editar/total/:matricula",
  verificarToken,
  AlunoController.editarTotal
);

router.patch(
  "/editar/parcial/:matricula",
  verificarToken,
  AlunoController.editarParcial
);

router.delete(
  "/excluir/todos",
  verificarToken,
  AlunoController.excluirTodos
);

router.delete(
  "/excluir/:matricula",
  verificarToken,
  AlunoController.excluirPorMatricula
);

export default router;
```

15. Arquivo principal

Arquivo: `src/index.js`

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import adminRouter from "../modules/admin/routes/admin.route.js";
import alunoRouter from "../modules/aluno/routes/aluno.route.js";

dotenv.config();

const app = express();

app.use(cors());
app.use(express.json());

app.get("/", function (requisicao, resposta) {
  return resposta.status(200).json({
    mensagem: "API do CRUD de alunos funcionando."
  });
});

app.use(adminRouter);
```

```
app.use(alunoRouter);

const porta = process.env.PORTA || 3000;

app.listen(porta, function () {
  console.log("Servidor executando na porta " + porta);
});
```

16. Rotas da API

Públicas

```
GET /
POST /admin/cadastrar
POST /admin/login
```

Protegidas

```
GET /admin/perfil
GET /listar
GET /listar/:matricula
POST /cadastrar
PUT /editar/total/:matricula
PATCH /editar/parcial/:matricula
DELETE /excluir/:matricula
DELETE /excluir/todos
```

17. Testes

Cadastro do administrador

```
POST http://localhost:3000/admin/cadastrar
Content-Type: application/json
```

```
{
  "nome": "Administrador",
  "email": "admin@email.com",
  "senha": "123456"
}
```

Login

```
POST http://localhost:3000/admin/login
Content-Type: application/json
```

```
{
  "email": "admin@email.com",
  "senha": "123456"
}
```

Resposta:

```
{
  "mensagem": "Login realizado com sucesso.",
  "token": "eyJhbGciOiJIUzI1NiIs...\"",
  "admin": {
    "id": 1,
    "nome": "Administrador",
    "email": "admin@email.com"
  }
}
```

Perfil autenticado

```
GET http://localhost:3000/admin/perfil
Authorization: Bearer SEU_TOKEN
```

Cadastrar aluno

```
POST http://localhost:3000/cadastrar
Authorization: Bearer SEU_TOKEN
Content-Type: application/json
```

```
{
  "matricula": "A12345678",
  "nome": "Maria da Silva",
  "email": "maria@email.com"
}
```

Listar alunos

```
GET http://localhost:3000/listar
Authorization: Bearer SEU_TOKEN
```

Editar totalmente

```
PUT http://localhost:3000/editar/total/A12345678
Authorization: Bearer SEU_TOKEN
Content-Type: application/json
```

```
{
  "nome": "Maria Souza",
  "email": "maria.souza@email.com"
}
```

Editar parcialmente

```
PATCH http://localhost:3000/editar/parcial/A12345678
Authorization: Bearer SEU_TOKEN
Content-Type: application/json
```

```
{
  "nome": "Maria Oliveira"
}
```

Excluir aluno

```
DELETE http://localhost:3000/excluir/A12345678
Authorization: Bearer SEU_TOKEN
```

18. Estrutura do frontend React

```
frontend/
├── src/
│   ├── components/
│   │   ├── AdminCadastroForm.jsx
│   │   ├── LoginForm.jsx
│   │   └── AlunoForm.jsx
```

```
├── AlunoLista.jsx
├── AlunoItem.jsx
├── services/
│   ├── api.js
│   ├── adminService.js
│   └── alunoService.js
├── App.jsx
├── main.jsx
├── index.css
├── .env
├── package.json
└── vite.config.js
```

19. Axios com JWT automático

```
npm install axios
```

Arquivo: `src/services/api.js`

```
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL
});

api.interceptors.request.use(function (configuracao) {
  const token = localStorage.getItem("token");

  if (token !== null) {
    configuracao.headers.Authorization =
      "Bearer " + token;
  }

  return configuracao;
});

export default api;
```

`.env` do frontend:

```
VITE_API_URL=http://localhost:3000
```

20. Serviço do administrador

Arquivo: `src/services/adminService.js`

```
import api from "../api";

export async function cadastrarAdmin(nome, email, senha) {
  const resposta = await api.post(
    "/admin/cadastrar",
    { nome, email, senha }
  );

  return resposta.data;
}

export async function fazerLogin(email, senha) {
  const resposta = await api.post(
    "/admin/login",
    { email, senha }
  );

  return resposta.data;
}

export async function buscarPerfil() {
  const resposta = await api.get("/admin/perfil");
  return resposta.data;
}
```

21. Serviço do aluno

Arquivo: `src/services/alunoService.js`

```
import api from "../api";

export async function listarAlunos() {
  const resposta = await api.get("/listar");
  return resposta.data;
}

export async function buscarAluno(matricula) {
  const resposta = await api.get("/listar/" + matricula);
  return resposta.data;
}

export async function cadastrarAluno(matricula, nome, email) {
  const resposta = await api.post(
    "/cadastrar",
    { matricula, nome, email }
  );
}
```

```
    return resposta.data;
  }

  export async function editarAluno(matricula, nome, email) {
    const resposta = await api.put(
      "/editar/total/" + matricula,
      { nome, email }
    );

    return resposta.data;
  }

  export async function excluirAluno(matricula) {
    const resposta = await api.delete(
      "/excluir/" + matricula
    );

    return resposta.data;
  }
}
```

22. Cadastro do administrador no React

Arquivo: `src/components/AdminCadastroForm.jsx`

```
import { useState } from "react";
import { cadastrarAdmin } from "../services/adminService";

function AdminCadastroForm({ onCadastroRealizado }) {
  const [nome, setNome] = useState("");
  const [email, setEmail] = useState("");
  const [senha, setSenha] = useState("");
  const [mensagem, setMensagem] = useState("");

  async function enviarFormulario(evento) {
    evento.preventDefault();

    try {
      const dados = await cadastrarAdmin(
        nome,
        email,
        senha
      );

      setMensagem(dados.mensagem);
      onCadastroRealizado();
    } catch (erro) {
      setMensagem(
        erro.response?.data?.mensagem ||
        "Erro ao cadastrar administrador."
      );
    }
  }
}
```

```
    }  
  }  
  
  return (  
    <form onSubmit={enviarFormulario}>  
      <h1>Cadastro do administrador</h1>  
  
      <input  
        type="text"  
        placeholder="Nome"  
        value={nome}  
        onChange={(evento) => setNome(evento.target.value)}  
      />  
  
      <input  
        type="email"  
        placeholder="E-mail"  
        value={email}  
        onChange={(evento) => setEmail(evento.target.value)}  
      />  
  
      <input  
        type="password"  
        placeholder="Senha"  
        value={senha}  
        onChange={(evento) => setSenha(evento.target.value)}  
      />  
  
      <button type="submit">Cadastrar</button>  
      <p>{mensagem}</p>  
    </form>  
  );  
}  
  
export default AdminCadastroForm;
```

23. Login no React

Arquivo: `src/components/LoginForm.jsx`

```
import { useState } from "react";  
import { fazerLogin } from "../services/adminService";  
  
function LoginForm({ onLoginRealizado, onMostrarCadastro }) {  
  const [email, setEmail] = useState("");  
  const [senha, setSenha] = useState("");  
  const [mensagem, setMensagem] = useState("");  
  
  async function enviarFormulario(evento) {  
    evento.preventDefault();  
  }  
}
```



```

    try {
      const dados = await fazerLogin(email, senha);
      onLoginRealizado(dados.token);
    } catch (erro) {
      setMensagem(
        erro.response?.data?.mensagem ||
        "Erro ao realizar login."
      );
    }
  }

  return (
    <form onSubmit={enviarFormulario}>
      <h1>Login</h1>

      <input
        type="email"
        placeholder="E-mail"
        value={email}
        onChange={(evento) => setEmail(evento.target.value)}
      />

      <input
        type="password"
        placeholder="Senha"
        value={senha}
        onChange={(evento) => setSenha(evento.target.value)}
      />

      <button type="submit">Entrar</button>

      <button
        type="button"
        onClick={() => onMostrarCadastro(true)}
      >
        Primeiro acesso
      </button>

      <p>{mensagem}</p>
    </form>
  );
}

export default LoginForm;

```

24. App.jsx

```

import { useState } from "react";
import AdminCadastroForm from "../components/AdminCadastroForm";

```

```
import LoginForm from "../components/LoginForm";
import AlunoLista from "../components/AlunoLista";

function App() {
  const tokenSalvo = localStorage.getItem("token");
  const [token, setToken] = useState(tokenSalvo);
  const [mostrarCadastro, setMostrarCadastro] = useState(false);

  function loginRealizado(novoToken) {
    localStorage.setItem("token", novoToken);
    setToken(novoToken);
  }

  function cadastroRealizado() {
    setMostrarCadastro(false);
  }

  function sair() {
    localStorage.removeItem("token");
    setToken(null);
  }

  if (token !== null) {
    return (
      <main>
        <button onClick={sair}>Sair</button>
        <AlunoLista />
      </main>
    );
  }

  if (mostrarCadastro === true) {
    return (
      <AdminCadastroForm
        onCadastroRealizado={cadastroRealizado}
      />
    );
  }

  return (
    <LoginForm
      onLoginRealizado={loginRealizado}
      onMostrarCadastro={setMostrarCadastro}
    />
  );
}

export default App;
```

25. Fluxo no frontend

```
LoginForm
  ↓
POST /admin/login
  ↓
Recebe token
  ↓
localStorage.setItem("token", token)
  ↓
Axios interceptor
  ↓
Authorization: Bearer TOKEN
  ↓
Backend valida
  ↓
CRUD liberado
```

Logout:

```
localStorage.removeItem("token");
```

26. Status HTTP utilizados

200 OK	operação realizada
201 Created	cadastro realizado
400 Bad Request	dados ausentes ou inválidos
401 Unauthorized	login/token inválido
403 Forbidden	acesso bloqueado
404 Not Found	registro não encontrado
409 Conflict	registro duplicado
500 Internal Server Error	erro interno

27. Sequência recomendada para as aulas

Aula 1 — Organização e PostgreSQL

- estrutura por módulos;
- conexão;
- tabelas;
- model.

Aula 2 — Cadastro do administrador

- validação;
- `bcrypt.hash`;

- `INSERT`;
- bloqueio do segundo cadastro.

Aula 3 — Login e JWT

- busca por e-mail;
- `bcrypt.compare`;
- `jwt.sign`;
- expiração.

Aula 4 — Middleware

- cabeçalho `Authorization`;
- padrão `Bearer`;
- `jwt.verify`;
- `next()`.

Aula 5 — CRUD protegido

- aplicar middleware;
- testar com e sem token;
- erros 401.

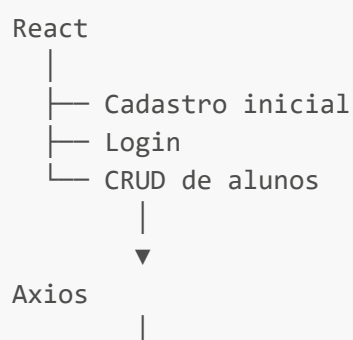
Aula 6 — React

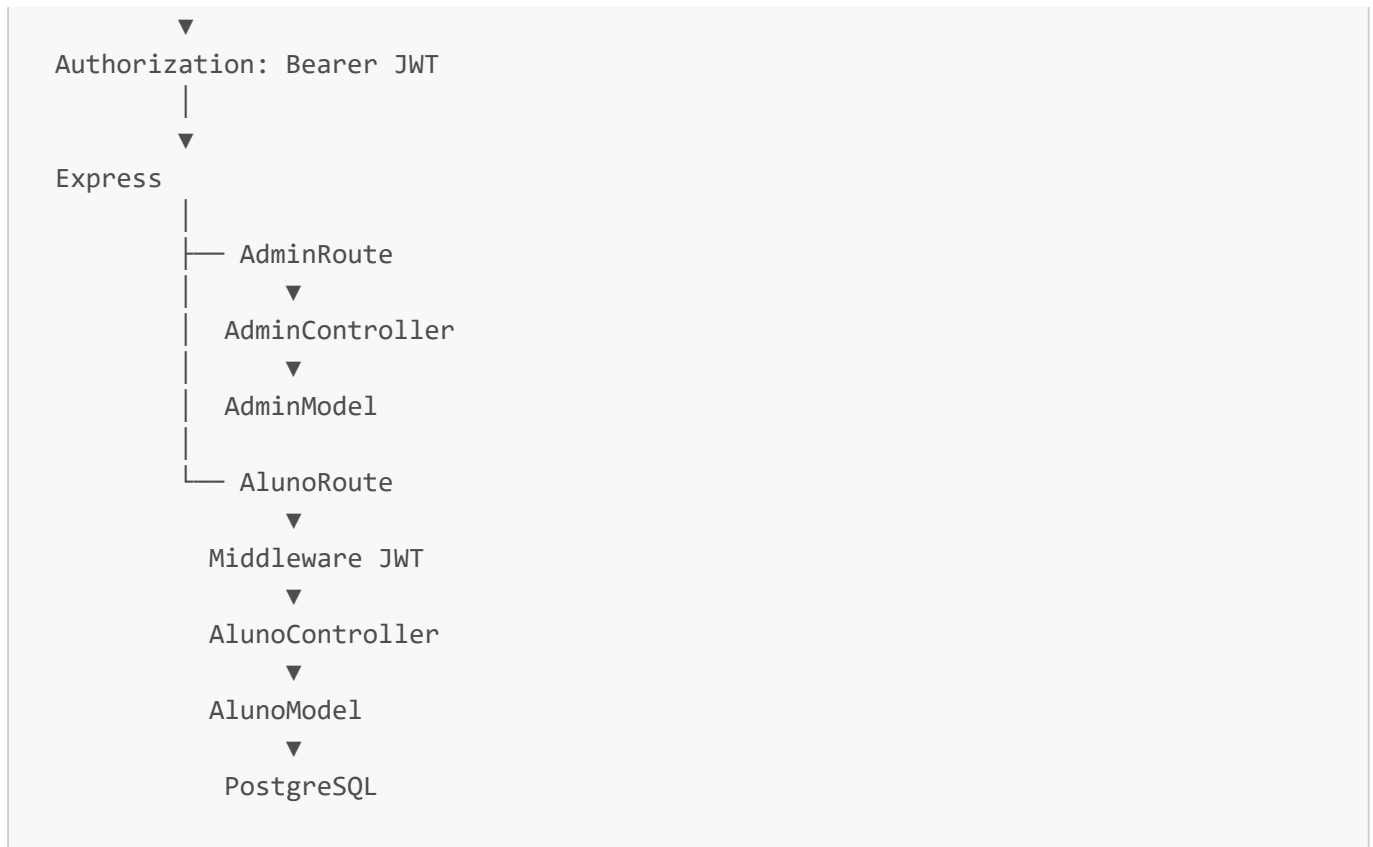
- cadastro inicial;
- login;
- `localStorage`;
- renderização condicional;
- logout.

Aula 7 — Axios

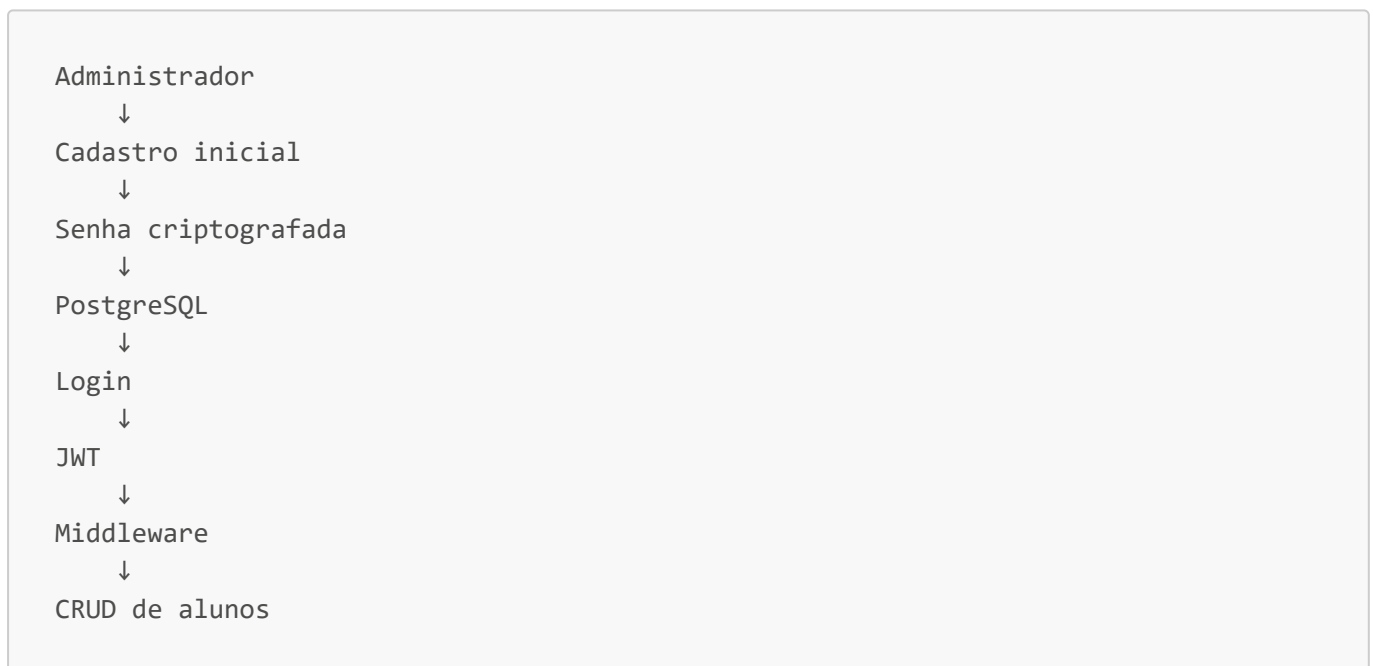
- `baseURL`;
- `services`;
- `interceptor`;
- JWT automático.

28. Arquitetura final

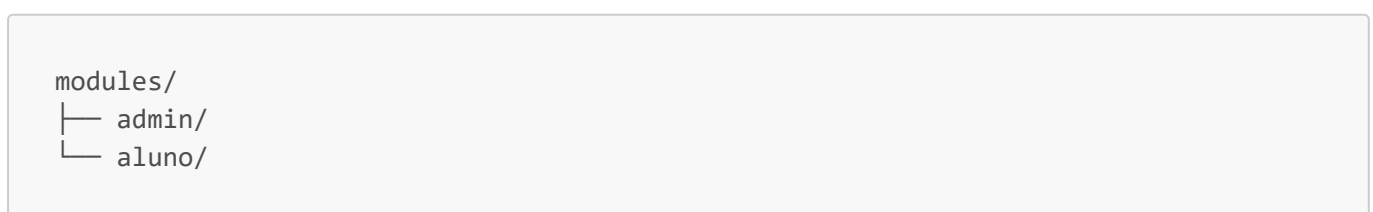




29. Resumo



A organização final é:



O administrador é a entidade autenticada. O aluno é a entidade administrada.